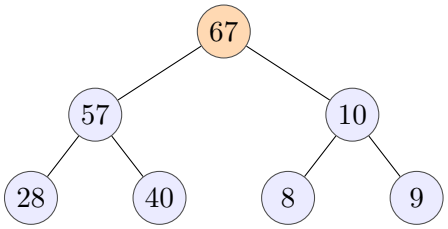
Benchmark Comparatif

Arbre binaire de recherche et tas

Tas max écrit à la main vs `priority_queue`

Insertion, recherche, suppression et extraction de la racine :
analyse de performance comparative



Auteur : BENNOUR RAFAA
Formation : ILISI – 2e année
Langage : C++17
Compilateur : g++ 15.2 `-O2`

Table des matières

1 Introduction **2**
- 1.1 Contexte et objectifs . . . 2
- 1.2 Les deux structures en image . . . 3
- 1.3 Jeux de données . . . 3
- 1.4 Opérations mesurées . . . 3
- 1.5 Complexités théoriques . . . 4

2 Méthodologie **5**
- 2.1 Environnement . . . 5
- 2.2 Protocole de mesure . . . 5
- 2.3 Implémentation . . . 5

3 Résultats : arbre binaire de recherche **8**
- 3.1 Démonstration . . . 8
- 3.2 Données brutes . . . 8
- 3.3 Visualisation . . . 9
- 3.4 Analyse . . . 9

4 Résultats : tas max à la main vs `priority_queue` **10**
- 4.1 Démonstration : tas max et tas min . . . 10
- 4.2 Données brutes . . . 11
- 4.3 Visualisation . . . 11
- 4.4 Analyse . . . 12

5 Analyse comparative approfondie **14**
- 5.1 ABR ou tas : chacun son usage . . . 14
- 5.2 Mémoire . . . 14
- 5.3 Tas à la main ou `priority_queue` ? . . . 15

6 Conclusion **16**
- 6.1 Synthèse des résultats . . . 16
- 6.2 Recommandations . . . 16
- 6.3 Conclusion finale . . . 16

Chapitre 1

Introduction

1.1 Contexte et objectifs

Ce rapport étudie deux structures en forme d'**arbre binaire** (chaque nœud a au plus deux enfants), qui rangent les données de deux façons différentes :

Arbre binaire de recherche (ABR)
pour chaque nœud, les valeurs plus petites sont à gauche et les plus grandes à droite. On peut donc **chercher** une valeur en descendant dans l'arbre, comme dans un dictionnaire.

Tas (heap)
chaque parent est plus grand (**tas max**) ou plus petit (**tas min**) que ses enfants. On ne cherche pas une valeur quelconque : on veut seulement obtenir très vite **le maximum** (ou le minimum), qui est toujours à la racine.

La partie principale du travail demandé porte sur le **tas max**, réalisé de deux manières :

1. **à la main, sans conteneur** : un simple tableau `int*` alloué avec `new`, et nos propres fonctions `monter` et `descendre` ;
2. **avec `std::priority_queue<int>`**, l'adaptateur de la STL, qui est un tas max par défaut.

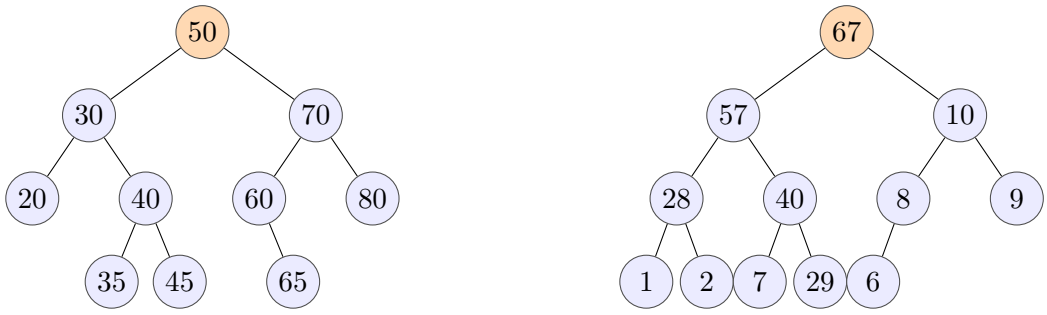
Les deux versions sont comparées sur les deux opérations d'un tas : **l'insertion** et **l'extraction de la racine** (la plus grande valeur).

1.1.1 Objectifs de l'étude

1. Implémenter un ABR complet : insertion, recherche, minimum, maximum, suppression (3 cas) et les quatre parcours.
2. Montrer l'effet de **l'ordre d'insertion** sur la hauteur de l'ABR, et donc sur sa vitesse.
3. Implémenter un tas max et un tas min sans conteneur.
4. Comparer le tas max écrit à la main avec `priority_queue` : qui insère le plus vite, et qui extrait la racine le plus vite ?

1.2 Les deux structures en image

Les mêmes valeurs ne donnent pas du tout le même arbre :



ABR : gauche $<$ nœud $<$ droite — Tas max : parent $\geq$ enfants

FIGURE 1.1 – À gauche l'ABR de la démonstration, à droite le tas max construit avec 7 9 67 1 28 8 10 29 2 57 40 6.

Un tas est toujours un arbre **complet** (rempli niveau par niveau, de gauche à droite). C'est pour cela qu'on peut le ranger dans un simple tableau, sans pointeurs :

67	57	10	28	40	8	9	1	2	7	29	6
0	1	2	3	4	5	6	7	8	9	10	11

Pour la case i : parent $= (i-1)/2$, enfant gauche $= 2i+1$, enfant droit $= 2i+2$.

1.3 Jeux de données

— **Démonstrations** : ABR avec 50 30 70 20 40 60 80 35 45 65 ; tas avec 7 9 67 1 28 8 10 29 2 57 40 6.

— **Benchmark ABR** : les clés $0, 1, \ldots, N-1$, insérées soit dans l'ordre **croissant**, soit dans un ordre **aléatoire** (mélange avec une graine fixe). N va jusqu'à 20 000 en ordre croissant (au-delà, c'est trop lent) et jusqu'à 1 000 000 en ordre aléatoire.

— **Benchmark tas** : $N = 10^3$ à 10^7 entiers, dans trois ordres : **aléatoire**, **croissant** et **décroissant**.

1.4 Opérations mesurées

Structure	Opération	Ce qu'on mesure
ABR	`inserer`	insérer les N clés
	`rechercher`	chercher chacune des N clés
	`supprimer`	supprimer les N clés une par une
Tas max	insertion	`inserer(x)` / `pq.push(x)`, N fois
	extraction de la racine	`extraireRacine()` / `top()` + `pop()`

Chaque résultat est un **temps moyen par opération**, en nanosecondes (ns).

1.5 Complexités théoriques

TABLE 1.1 – Complexités (h = hauteur de l'ABR, n = nombre d'éléments).

Opération	ABR équilibré	ABR dégénéré (liste)	Tas
Insertion	$O(\log n)$	$O(n)$	$O(\log n)$
Recherche d'une valeur	$O(\log n)$	$O(n)$	$O(n)$
Lire le maximum	$O(\log n)$	$O(n)$	$O(1)$ (tas max)
Supprimer / extraire le maximum	$O(\log n)$	$O(n)$	$O(\log n)$

Pour l'ABR, tout dépend de la **hauteur** h : chaque opération coûte $O(h)$. Si les valeurs arrivent dans un ordre aléatoire, h reste proche de $\log_2 n$; si elles arrivent triées, chaque nouvelle valeur va à droite de la précédente et l'arbre devient une **liste** : $h = n$. Le tas, lui, est toujours complet : sa hauteur vaut toujours $\lfloor \log_2 n \rfloor$, quel que soit l'ordre.

Chapitre 2

Méthodologie

2.1 Environnement

— Processeur : Intel Core i7-8550U à 1,8 GHz (4 cœurs, 8 threads), cache L2 de 256 Ko par cœur, cache L3 de 8 Mo.

— Mémoire : 8 Go de DDR4 à 2400 MHz.

— Système : Windows 11 Pro ; compilateur g++ 15.2 (MinGW-w64), options `-O2 -std=c++17`.

— C'est un ordinateur portable : sa fréquence varie (mode Turbo, température). Pour le vérifier, le benchmark du tas a été lancé deux fois. Les temps changent de 10 à 30 % d'une série à l'autre, mais les tendances restent les mêmes. Les tableaux montrent la première série.

2.2 Protocole de mesure

— Chronomètre : `std::chrono::steady_clock`.

— Chaque mesure est répétée (5 fois pour le tas, 3 fois pour l'ABR et pour $N = 10^7$) et l'on garde **le meilleur temps**, le moins perturbé par le système.

— Les deux versions du tas reçoivent **exactement les mêmes données**.

— Les valeurs extraites sont additionnées dans une variable `volatile` pour que le compilateur ne supprime pas le calcul.

— **Vérification** avant les mesures : sur 100 000 valeurs, le tas écrit à la main et `priority_queue` doivent sortir les mêmes valeurs, dans l'ordre décroissant. Pour l'ABR, chaque recherche doit réussir et chaque suppression doit trouver sa clé.

2.3 Implémentation

2.3.1 Arbre binaire de recherche

Un nœud contient une valeur et deux pointeurs. L'insertion et la recherche sont **itératives** (une boucle, pas de récursion) : en ordre croissant l'arbre a une hauteur de 20 000, et une fonction récursive risquerait de saturer la pile.

```
struct Noeud { int valeur; Noeud* gauche = nullptr; Noeud* droite = nullptr; };

bool rechercher(int v) const {
    Noeud* c = racine;
    while (c != nullptr) {
        if (v == c->valeur) return true;
        c = (v < c->valeur) ? c->gauche : c->droite;   // gauche si plus petit
    }
    return false;
}
```

La **suppression** traite trois cas :

1. le nœud est une **feuille** : on l'enlève ;
2. il a **un seul enfant** : l'enfant prend sa place ;
3. il a **deux enfants** : on copie dans le nœud le **minimum du sous-arbre droit** (la valeur juste au-dessus), puis on supprime ce minimum, qui n'a jamais d'enfant gauche.

```
Noeud* n = *p;                        // p : adresse du lien qui pointe vers le noeud
if (n->gauche == nullptr)      { *p = n->droite; delete n; }   // cas 1 et 2
else if (n->droite == nullptr) { *p = n->gauche; delete n; }   // cas 2
else {                                                         // cas 3
    Noeud** q = &n->droite;
    while ((*q)->gauche != nullptr) q = &(*q)->gauche;         // minimum à droite
    Noeud* m = *q;
    n->valeur = m->valeur;
    *q = m->droite;
    delete m;
}
```

Les parcours préfixe, infixe et suffixe sont récursifs ; le parcours en largeur utilise une file (`queue`).

2.3.2 Tas max sans conteneur

Le tas est un tableau `int* t` géré à la main : quand il est plein, on alloue un tableau deux fois plus grand et on recopie (`agrandir`).

```
void inserer(int v) {                  // ajouter à la fin, puis faire monter
    if (n == capacite) agrandir();
    t[n] = v;
    int i = n++;
    while (i > 0 && t[i] > t[(i-1)/2]) { swap(t[i], t[(i-1)/2]); i = (i-1)/2; }
}

int extraireRacine() {                 // le dernier remplace la racine, puis descend
    int max = t[0];
    t[0] = t[--n];
    descendre(0);                      // échange avec le plus grand enfant
    return max;
}
```

2.3.3 Tas min

Le tas min est le même code avec les comparaisons inversées (`<` au lieu de `>`) : chaque parent est plus petit que ses enfants, et la racine est le minimum.

2.3.4 Tas max avec `priority_queue`

```cpp
#include <queue>
priority_queue<int> pq;                 // tas max par défaut (un vector en dessous)
for (int x : donnees) pq.push(x);                       // insertion
while (!pq.empty()) { s += pq.top(); pq.pop(); }        // extraction de la racine
```

2.3.5 Organisation du projet

Fichier	Rôle
`ArbreBinaireRecherche.h`	classe `ABR`
`TasMax.h`, `TasMin.h`	tas écrits à la main (tableau `new`/`delete`)
`DemoABR.cpp`, `DemoTas.cpp`	démonstrations sur de petits exemples
`BenchmarkABR.cpp`	ordre croissant vs aléatoire
`BenchmarkTas.cpp`	tas à la main vs `priority_queue`

Chapitre 3

Résultats : arbre binaire de recherche

3.1 Démonstration

```
Nombre de noeuds : 10      Hauteur : 4      Minimum : 20      Maximum : 80
Parcours prefixe (racine, gauche, droite) : 50 30 20 40 35 45 70 60 65 80
Parcours infixe  (gauche, racine, droite) : 20 30 35 40 45 50 60 65 70 80
Parcours suffixe (gauche, droite, racine) : 20 35 45 40 30 65 60 80 70 50
Parcours en largeur (niveau par niveau)   : 50 30 70 20 40 60 80 35 45 65
Recherche de 45 : trouve      Recherche de 55 : absent
Insertion de 40 (deja present) : refuse
Suppression de 20 (feuille)           -> 30 35 40 45 50 60 65 70 80
Suppression de 60 (un seul enfant 65) -> 30 35 40 45 50 65 70 80
Suppression de 50, la racine (deux enfants, remplacee par 65) -> 30 35 40 45 65 70 80
```

Le parcours **infixe donne toujours les valeurs triées** : c'est la propriété qui définit un ABR. Après chaque suppression, l'infixe reste trié, ce qui prouve que l'arbre est resté correct.

3.2 Données brutes

Table 3.1 – ABR, insertion dans l'ordre **croissant** (ns par opération).

N	Hauteur	Insertion	Recherche	Suppression
1 000	1 000	1 866	2 045	920
2 000	2 000	4 441	5 752	2 223
5 000	5 000	11 475	14 627	6 572
10 000	10 000	23 921	29 734	15 482
20 000	20 000	48 991	65 438	31 689

TABLE 3.2 – ABR, insertion dans un ordre **aléatoire** (ns par opération).

N	Hauteur	$\lceil \log_2 N \rceil$	Insertion	Recherche	Suppression
1 000	22	10	144.2	38.1	104.5
2 000	23	11	131.6	43.4	111.3
5 000	29	13	169.8	62.0	135.6
10 000	29	14	265.8	93.0	190.2
20 000	33	15	338.9	150.2	274.2
50 000	38	16	352.2	163.7	288.3
100 000	41	17	462.2	180.0	331.2
1 000 000	50	20	1 049.9	720.0	979.8

3.3 Visualisation

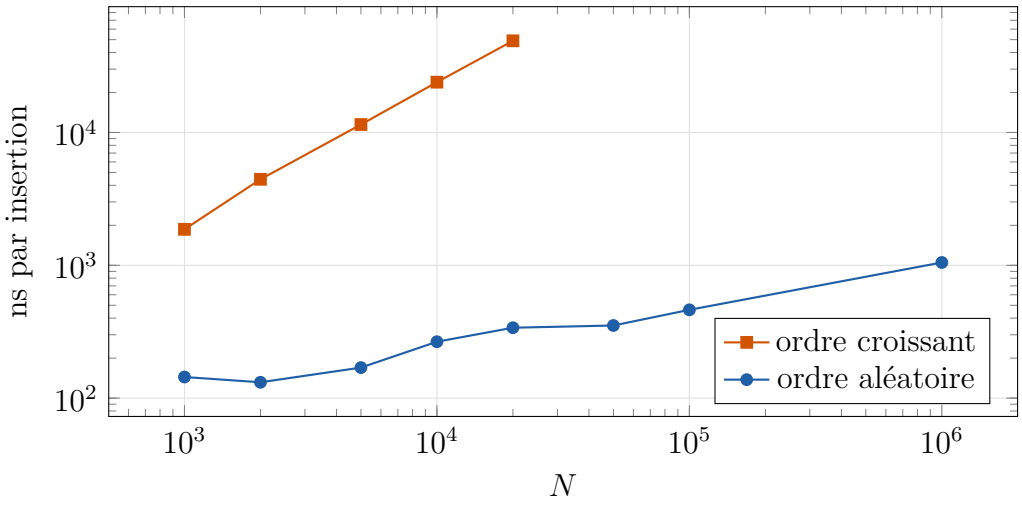


FIGURE 3.1 – Temps d'insertion dans l'ABR (échelles logarithmiques).

3.4 Analyse

— **Ordre croissant** : la hauteur est égale à N, l'arbre est une liste. Quand N double, le temps par opération double aussi (un peu plus, de 2 à 2,4 fois) : chaque opération est en $O(n)$, et remplir l'arbre coûte $O(n^2)$. À $N = 20\,000$, l'insertion est **145 fois** plus lente qu'en ordre aléatoire (48 991 contre 339 ns).

— **Ordre aléatoire** : la hauteur reste petite (50 pour un million de nœuds, environ $2{,}5 \times \log_2 N$). Le temps reste très petit par rapport à l'ordre croissant, mais il augmente **plus vite que la hauteur** : de 10^3 à 10^6, la hauteur est multipliée par 2,3 et le temps d'insertion par 7.

— La raison est le cache. Un nœud occupe environ 32 octets : à $N = 10^5$ l'arbre fait environ 3 Mo et tient dans le cache L3 (8 Mo) ; à $N = 10^6$ il fait environ 32 Mo et n'y tient plus. Le temps d'insertion passe alors de 462 à 1 050 ns alors que la hauteur n'augmente que de 41 à 50 : les nœuds sont dispersés en mémoire, donc chaque descente vers un enfant est un accès lent à la mémoire.

Chapitre 4

Résultats : tas max à la main vs `priority_queue`

4.1 Démonstration : tas max et tas min

```
===== TAS MAX =====
Tableau : 67 57 10 28 40 8 9 1 2 7 29 6
Racine (maximum) : 67
Extractions successives : 67 57 40 29 28 10 9 8 7 6 2 1

===== TAS MIN =====
Tableau : 1 2 6 7 28 8 10 29 9 57 40 67
Racine (minimum) : 1
Extractions successives : 1 2 6 7 8 9 10 28 29 40 57 67
```

Extraire la racine plusieurs fois donne les valeurs **triées** : dans l'ordre décroissant pour le tas max, croissant pour le tas min (c'est le principe du tri par tas). Le tableau, lui, n'est pas trié : seule la règle parent/enfants est respectée.

4.2 Données brutes

TABLE 4.1 – Tas max : temps moyen par opération (ns). En vert, la version la plus rapide.

		Insertion		Extraction de la racine	
Données	N	À la main	`priority_queue`	À la main	`priority_queue`
aléatoire	10^3	20.1	26.4	86.7	50.1
	10^4	25.9	28.1	103.2	55.4
	10^5	24.7	26.7	147.3	81.1
	10^6	23.9	22.3	229.6	187.4
	10^7	28.7	26.8	399.9	624.4
croissant	10^3	16.4	19.4	64.3	29.8
	10^4	26.3	31.1	74.4	47.6
	10^5	44.5	50.1	107.6	79.4
	10^6	44.0	55.8	106.1	90.2
	10^7	50.1	64.7	126.5	107.6
décroissant	10^3	6.6	8.2	81.4	49.7
	10^4	15.0	14.7	108.4	66.4
	10^5	11.4	9.3	106.8	66.2
	10^6	7.0	6.0	120.0	99.6
	10^7	9.2	7.3	145.1	121.6

4.3 Visualisation

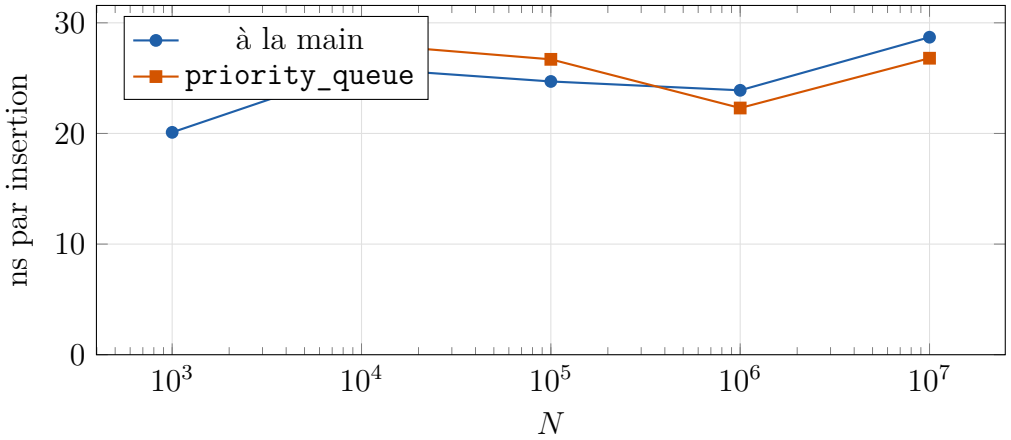


FIGURE 4.1 – Insertion, données aléatoires : la version à la main est devant jusqu'à 10^5, puis les deux sont presque égales.

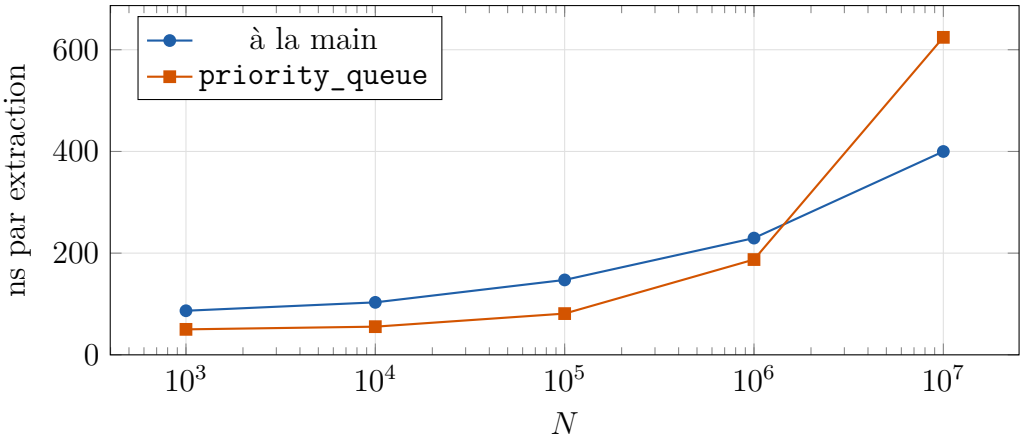


FIGURE 4.2 – Extraction de la racine, données aléatoires : `priority_queue` est devant jusqu'à 10^6, les courbes se croisent entre 10^6 et 10^7.

4.4 Analyse

4.4.1 Insertion : la version à la main gagne le plus souvent

Le tas écrit à la main est plus rapide dans **9 cas sur 15** :

— en ordre **croissant**, il gagne pour toutes les tailles, de 11 à 23 % ;

— en ordre **aléatoire**, il gagne jusqu'à 10^5 (de 7 à 24 %). À 10^6 et 10^7, `priority_queue` est un peu devant (environ 7 %), mais ce n'est pas stable : dans la deuxième série, le tas à la main gagne à 10^6 et les deux sont à égalité à 10^7 ;

— en ordre **décroissant**, les temps sont très petits (6 à 15 ns) et les écarts sont du même ordre que les variations d'une série à l'autre : on ne peut pas désigner de gagnant.

Les deux font le même travail (ajouter à la fin puis faire monter), mais notre version est plus simple :

— `priority_queue::push` passe par `vector::push_back` puis par l'algorithme général `push_heap`, écrit pour n'importe quel type et n'importe quel comparateur ;

— notre `inserer` travaille directement sur un tableau d'`int`, avec une comparaison `>` écrite en dur.

L'ordre des données compte beaucoup plus que la version choisie :

— **décroissant** (6 à 15 ns) : chaque nouvelle valeur est plus petite que son parent, elle ne monte pas du tout. C'est le **meilleur cas** : $O(1)$ par insertion.

— **croissant** : chaque nouvelle valeur est la plus grande et monte **jusqu'à la racine**. C'est le **pire cas** : $\log_2 n$ échanges à chaque fois. On le voit bien à partir de 10^5 (44 à 50 ns contre 24 à 29 ns en aléatoire). À $N = 10^3$, l'ordre croissant est pourtant un peu plus rapide que l'aléatoire (16 contre 20 ns) : probablement parce que la comparaison est toujours vraie, donc le processeur la prévoit parfaitement, alors qu'en aléatoire il se trompe souvent.

— **aléatoire** : en moyenne, une valeur ne monte que d'un ou deux niveaux. L'insertion coûte donc presque $O(1)$ en moyenne, ce qui explique les temps presque constants.

4.4.2 Extraction de la racine : `priority_queue` gagne presque partout

— `priority_queue` est plus rapide dans **14 cas sur 15**. Pour les petits tas l'écart est grand : 50,1 contre 86,7 ns à $N = 10^3$ en aléatoire, soit 42 % de moins, et 29,8 contre 64,3 ns en croissant.

— En aléatoire, l'écart reste autour de 45 % jusqu'à 10^5, puis il diminue nettement : à 10^6, 187 contre 230 ns (18 %).

— Seule exception : à $N = 10^7$ en aléatoire, le tas à la main est devant (400 contre 624 ns). Ce résultat se répète dans la deuxième série (468 contre 812 ns), ce n'est donc pas une erreur de mesure isolée.

Pourquoi `priority_queue` est-elle plus rapide ? `pop_heap` de la bibliothèque standard (libstdc++) n'utilise pas la même méthode que notre `descendre` :

Notre `descendre`	`pop_heap` de la STL
On place le dernier élément à la racine, puis on l'**échange** avec le plus grand enfant, niveau après niveau.	On crée un **trou** à la racine et on y fait remonter le plus grand enfant, jusqu'en bas ; puis on place le dernier élément dans le trou et on le fait remonter un peu.
2 comparaisons par niveau + 1 échange (3 écritures)	1 comparaison par niveau + 1 simple copie
S'arrête dès que l'élément est bien placé	Descend toujours jusqu'en bas

L'élément venu de la fin du tableau est petit : il redescend presque toujours tout en bas. Le test « est-ce que je m'arrête ? » de notre version est donc presque toujours inutile, et la méthode du trou fait moins de travail par niveau.

Sur cette machine, avec g++ 15.2, ce travail en moins suffit à rendre `priority_queue` plus rapide même pour les petits tas, qui tiennent entièrement dans le cache.

Et l'exception à 10^7 ? À $N = 10^7$, le tas occupe 40 Mo, cinq fois plus que le cache L3 (8 Mo, soit environ deux millions d'`int`). Mais la taille seule n'explique pas l'exception : en ordre croissant et décroissant, le tas fait aussi 40 Mo à $N = 10^7$, et `priority_queue` reste devant.

Ce qui change avec des données aléatoires, c'est que le chemin suivi à chaque extraction est imprévisible : dans le bas de l'arbre, chaque niveau coûte un accès lent à la mémoire et une erreur de prédiction du processeur. Le temps dépend alors surtout de ces accès, et les deux méthodes en font à peu près autant : l'avantage de la méthode du trou disparaît. Cela explique que l'écart se réduise dès 10^6, mais pas pourquoi `priority_queue` devient **nettement plus lente** à 10^7. Nous n'avons pas d'explication vérifiée pour ce dernier point : il faudrait un outil de profilage pour compter les accès mémoire de chaque version.

Chapitre 5

Analyse comparative approfondie

5.1 ABR ou tas : chacun son usage

Table 5.1 – Comparaison des deux structures.

	ABR	Tas
Règle	gauche $<$ nœud $<$ droite	parent $\geq$ enfants (tas max)
Forme	dépend de l'ordre d'insertion	toujours complète
Hauteur	entre $\log_2 n$ et n	toujours $\lfloor \log_2 n \rfloor$
Chercher une valeur	oui, $O(h)$	non (il faut tout parcourir)
Obtenir le maximum	descendre à droite, $O(h)$	la racine, $O(1)$
Valeurs triées	parcours infixe	extractions successives
Stockage	nœuds + pointeurs	un tableau
Usage typique	dictionnaire, ensemble trié	file de priorité, tri par tas, Dijkstra

L'ordre d'insertion est le point faible de l'ABR simple (145 fois plus lent en ordre croissant) ; le tas n'a pas ce problème, car il reste toujours complet. Pour avoir un arbre de recherche toujours rapide, il faut un arbre **équilibré** (AVL, arbre rouge-noir) : c'est ce qu'utilise `std::map`.

5.2 Mémoire

— **ABR** : chaque nœud contient un `int` (4 octets) et deux pointeurs (2×8 octets), soit 24 octets avec l'alignement, plus le surcoût de `new` : environ **32 octets par valeur**. Les nœuds sont dispersés en mémoire.

— **Tas** : **4 octets par valeur**, rangées les unes à côté des autres. Le tableau double quand il est plein, donc au pire la moitié des cases est vide.

— Le tas à la main et `priority_queue` utilisent **la même mémoire** : notre tableau double comme le `vector` interne de `priority_queue`.

Un tas utilise donc environ 8 fois moins de mémoire qu'un ABR pour les mêmes valeurs, et il profite beaucoup mieux du cache.

5.3 Tas à la main ou `priority_queue` ?

Table 5.2 – Bilan de la comparaison.

Critère	À la main	`priority_queue`
Insertion	plus rapide le plus souvent (9 cas sur 15)	
Extraction de la racine		plus rapide (14 cas sur 15)
Extraction, $N = 10^7$ aléatoire	plus rapide	
Quantité de code	une classe entière	une ligne
Risque d'erreur	`new`/`delete`, indices	code déjà testé
Changer d'ordre (tas min)	réécrire les comparaisons	`greater<int>`
Comprendre le fonctionnement	tout est visible	caché

Chapitre 6

Conclusion

6.1 Synthèse des résultats

1. L'ABR fonctionne correctement : le parcours infixe reste trié après les trois cas de suppression.
2. Sa vitesse dépend de sa hauteur : avec des valeurs triées, il devient une liste et il est **145 fois plus lent** à $N = 20\,000$.
3. Le tas reste toujours complet, donc toujours rapide ; il donne le maximum (ou le minimum) en $O(1)$.
4. Le tas max écrit à la main **insère plus vite** que `priority_queue` dans 9 cas sur 15 (de 7 à 24 %), surtout avec des données croissantes ; pour les données décroissantes, les écarts sont trop faibles pour conclure.
5. Pour **l'extraction de la racine**, `priority_queue` est plus rapide dans 14 cas sur 15 (jusqu'à 42 % de moins), grâce à la méthode du trou, qui fait moins de travail par niveau. Seule exception : $N = 10^7$ en données aléatoires, où le tas à la main est devant ; nous n'en avons pas d'explication vérifiée.

6.2 Recommandations

— Pour **chercher** des valeurs : un ABR, mais équilibré (`std::set`, `std::map`) si l'ordre des données n'est pas aléatoire.

— Pour obtenir sans cesse **le plus grand** (ou le plus petit) : un tas.

— Dans un vrai programme : `priority_queue`, plus courte, plus sûre, et plus rapide pour l'extraction.

— Écrire le tas à la main reste utile pour **apprendre** comment il fonctionne ; son seul vrai avantage mesuré ici est une insertion un peu plus rapide.

6.3 Conclusion finale

L'ABR et le tas sont deux arbres binaires, mais ils ne répondent pas à la même question : l'ABR permet de **chercher** n'importe quelle valeur, le tas donne seulement **la plus grande**, mais toujours en temps constant. Pour le tas max, le code écrit à la main insère un peu plus vite, mais la technique plus fine de la bibliothèque standard

l'emporte nettement pour l'extraction. Les mesures dépendent aussi de la machine et du compilateur : sur un autre PC, avec g++ 13, le tas à la main extrayait plus vite jusqu'à un million d'éléments. Aucune des deux versions n'est la plus rapide partout : le résultat dépend de l'opération, de la taille, de l'ordre des données, de la machine et du compilateur. C'est pour cela qu'il faut mesurer plutôt que deviner.